Senior Engineer Take-Home Exercise

Vitarc

Role	Senior Software Engineer
Timebox	8 hours for the exercise
Submission	A link to a private Git repository
Stack	Your choice
Interview	Scheduled within 48 hours of submission. Video call, screen share.

Thank you for taking the time to work through this exercise. The goal is to give us a realistic view of how you approach an ambiguous, real-world engineering problem, and to give you a realistic view of the work you would be doing at Vitarc. We have tried to make the exercise genuinely representative rather than abstract.

Vitarc is building a chronic disease companion. Patients typically have multiple conditions managed across multiple hospitals, with data arriving from wearables, clinical systems, and the patients themselves. Our job is to turn all of this into something a patient and their care team can act on. Almost every engineering decision in the product eventually touches the ingestion and reasoning layer. That is what this exercise is about.

You will be given data for one patient, Sara Shalabi, a 55-year-old woman with hypertension, type 2 diabetes, and mild asthma. Her data spans three hospitals, a wearable device, and her own app entries over a 48-hour window. Your task is to build a backend service that ingests this data, stores it in a unified model, and exposes it through an API with anomaly detection.

The exercise is intentionally underspecified in places. We are more interested in the decisions you make than in whether you meet a precise specification. Where you have to make a judgment call, make one and explain it in your write-up. Where you choose not to build something, tell us why.

The data

You will receive seven files alongside this document. A brief description of each is below. You should expect these to reflect real world messiness:

- inconsistent units,
- duplicate readings,
- timestamps in mixed timezones,
- devices that disconnect,
- and PDF documents that vary in structure from one hospital to another.

Handling this is part of the exercise.

patient_profile.json contains demographics, active conditions with ICD-10 codes, current medications with scheduled dosing times, allergies, and the patient's care team across three hospitals. This is the reference document for everything else.

wearable_export.xml is an Apple Health export covering 48 hours. It contains heart rate, SpO_2, step count, heart rate variability, and sleep analysis records. Each metric is a separate record type. Heart rate is an integer beats-per-minute; SpO_2 is a decimal between 0 and 1. Timestamps use the +0300 offset. This is the raw format you would receive from a real Apple Watch export.

Note: We are aware that healthkit integration for the applewatch will not send in the raw xml but instead a structured JSON we are using it to simulate the complexity of multiformat ingestion.

manual_entries.json is the patient's self-reported data over the same window: blood pressure readings, blood glucose measurements, medication confirmations, symptom reports, and inhaler use. This is what a patient would enter through the app.

The following pdfs and jpeg was run through claude to remove patient data and replicate the format of existing hospital documents:

lab_cbc_kauh.pdf is a complete blood count report from King Abdulaziz University Hospital. Table-based format with bilingual Arabic and English headers.

chest_xray_kauh.jpeg is a chest X-ray report from the same hospital. Structured radiology format with distinct sections for findings and impression.

renal_ultrasound_sgh.pdf is a renal Doppler ultrasound report from Saudi German Hospital. Different layout and structure from the KAUH reports.

echocardiogram_fakeeh.pdf is an echocardiogram report from Dr. Soliman Fakeeh Hospital. Includes both a measurements table and narrative findings. Third distinct format.

What we'd like you to build

A backend service with three responsibilities. We are not prescribing an architecture, a data model, or a language. Those choices are part of what we are evaluating.

1. Ingestion

Parse all seven data sources and store them in a unified patient timeline. This means handling Apple Health's XML format with its per-record-type structure, parsing the manual entries JSON, and extracting structured data from the PDFs. At a minimum, the PDF extraction should capture the quantitative lab values and the key clinical findings.

The hospital documents come in 3 different layouts, which is intentional. Your pipeline should be able to handle format variation rather than being pinned to a single template.\

2. Health snapshot API

Expose an endpoint that returns the patient's current health snapshot. The shape of the response is up to you, but it should make it possible to answer, at minimum, the following questions about the patient:

- What are her most recent vitals, and when and how were they captured?
- Has she been taking her medications as prescribed over the last 48 hours?
- What symptoms has she reported?
- What are the most clinically relevant findings from her hospital records?
- What should her care team be paying attention to right now?

3. Anomaly detection

Identify anomalies in the patient's data and surface them through the snapshot. At a minimum, the system should be able to detect vitals outside of clinically relevant thresholds, missed or significantly delayed medications, trends over time (a vital moving consistently in one direction across multiple readings), and clinically significant findings in the documents. What counts as an anomaly, whether thresholds should be global or patient-specific, how you prioritize multiple concurrent anomalies, and whether you can correlate signals across sources are all decisions you get to make.

Out of scope

Please do not spend time on any of the following. We are not evaluating them in this exercise. No user interface or frontend. No authentication or user management. No deployment; local is fine. No LLM or AI integration; we will discuss how you would approach that in the interview. No performance optimization beyond what is natural; the dataset is small.

What to submit

We would like four things, in order of importance:

1. **The code,** with a README that explains how to run it locally and how to exercise the API. A CLI demonstration, a test script, or a Postman collection is fine. A UI is not expected.
2. **A short write-up** of no more than two pages. We would like this to cover the key architectural decisions you made and why, the tradeoffs you considered, the things you chose not to build and why, and a paragraph on how you would integrate an LLM or agentic component into this pipeline if this were a real system.
3. **A note on scale.** A paragraph or diagram describing how your design would evolve from serving 1 patient to serving 10000 concurrent patients. You do not need to implement this.
4. **The Git history.** We find commit history informative. Please do not squash before submitting.

How we'll evaluate this

We are not grading against a checklist. We will read your code, your write-up, and your commit history, and form a view on how you think about building real systems. The aspects we pay the most attention to are the following:

Data modeling and schema design. How do you represent heterogeneous medical data coherently? Do you normalize units and timestamps? Do your abstractions make it obvious where a new data source would plug in?

How you handle messy data. Real medical data contains duplicates, gaps, out-of-order records, and inconsistent formats. We pay attention to how your pipeline behaves on the rough edges of the input.

Judgment under ambiguity. Much of this exercise is underspecified. We care more about the quality of your decisions. Whether your write up makes them explicit, whether the tradeoffs are articulated, whether you recognized the ambiguity in the first place, than about arriving at any right answer.

Clinical awareness. The work has human consequences. We are not looking for medical expertise, but we are looking for engineering decisions that reflect an understanding of what this data represents: validation, conservative behavior on uncertainty, clear attribution of source and time for every reading.

Prioritization. 8 hours is not enough to build everything. What you chose to build tells us more than a complete but mediocre submission would.

The interview

The follow up interview is a 60 minute conversation over video, with screen share. It has three rough parts, though they run together in practice.

In the 1st part, you walk us through your submission, explaining the decisions you made. We will ask questions throughout.

In the 2nd part, we will present a handful of scenarios and ask how your system would handle them, or how you would modify it to handle them. For example: how would the system behave if a device disconnects for several hours, how would you onboard a new hospital whose lab reports are in a different language, how would you extend the system to prioritize between multiple concurrent anomalies.

In the 3d part, we will ask you to add a small feature live, talking through your thinking as you code. The feature will involve combining signals across data sources in a way that is relevant to the domain. We do not expect a polished solution in twenty minutes.

Notes

Clarifying questions. You are welcome to ask us clarifying questions before you start.

AI tools. You may use AI assistants during the exercise.

Sample data. The data belongs to a fictional patient. It is designed to resemble real clinical data closely enough to be useful for this exercise, but all identifiers, names, and details are synthetic.